

Все о логическом выводе в Transformer

Часть 7 из [Как масштабировать свою модель \(Часть 6: Обучение LLaMA | Часть 8: Использование LLaMA\)](#)

Выводы, которые можно сделать на основе Transformer, могут сильно отличаться от результатов обучения. Отчасти это связано с тем, что при выводе необходимо учитывать новый фактор — задержку. В этом разделе мы рассмотрим весь процесс — от выборки одного нового токена из модели до эффективного масштабирования большого Transformer на множестве ускорителей в рамках механизма вывода.

АВТОРЫ
[Джейкоб Остин](#)
[Шолто Дуглас](#)
[Рой Фростиг](#)
[Ансельм Левская](#)
[Чарли Чен](#)
[Шарад Викрам](#)

[Федерико Леброн](#)
[Питер Чой](#)
[Винай Рамасеш](#)
[Альберт Уэбсон](#)
[Райнер Поуп*](#)

МЕСТО РАБОТЫ
Google DeepMind

опубликовано
4 февраля 2025 года

Основы логического вывода в трансформерах

Итак, вы обучили трансформер и хотите использовать его для генерации новых последовательностей. В конце концов, рост показателей и снижение кривых потерь — это лишь косвенные признаки того, что что-то интересное произойдет, когда дело дойдет до практики!¹

Сэмплирование концептуально просто. Мы вводим последовательность, и наш любимый трансформер выдает результат. $\log p(\text{next token} | \text{previous tokens})$, то есть логарифмические вероятности для всех возможных следующих токенов. Мы можем сгенерировать выборку из этого распределения и получить новый токен. Добавим этот токен и повторим процесс, чтобы получить последовательность токенов, которая является продолжением подсказки.

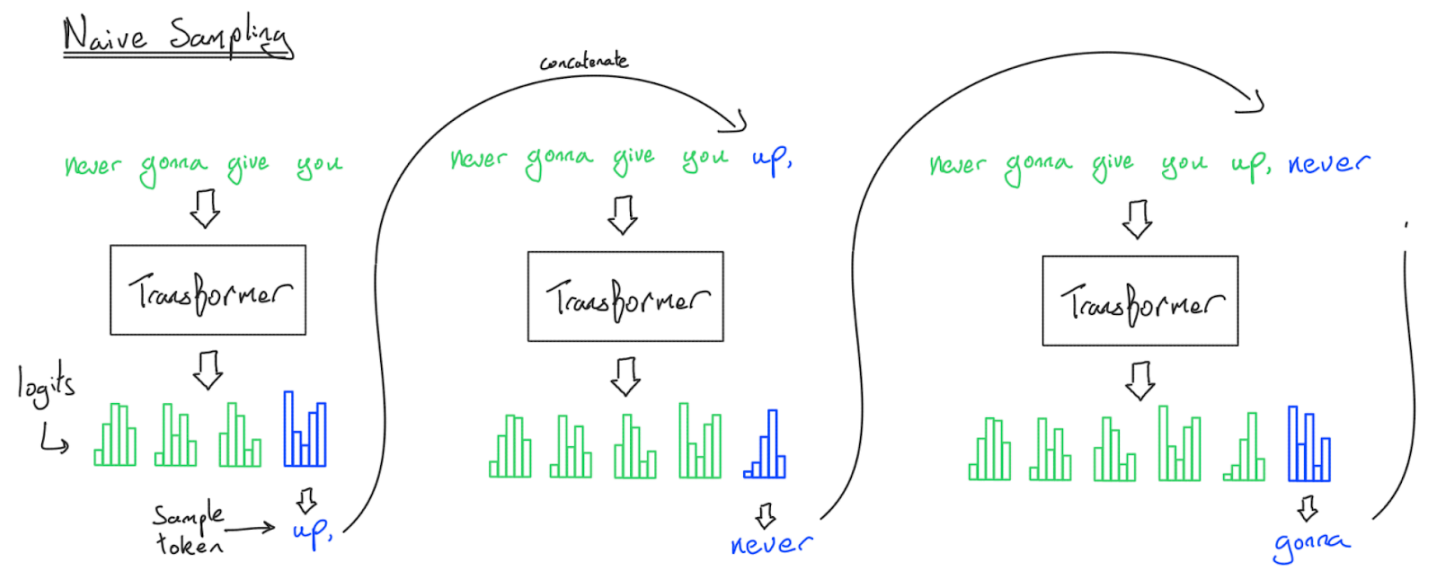


Рисунок: наивная выборка из Transformer. Синие logits дают нам распределение по следующему токенов, из которого мы можем выбрать один. Обратите внимание, что на каждом шаге обрабатывается весь префикс, что приводит к увеличению времени выполнения алгоритма.

Мы только что описали наивную реализацию выборки Transformer, и хотя она работает, **на практике мы никогда ее не используем**, потому что каждый раз при генерации токена мы заново обрабатываем всю последовательность. Этот алгоритм $O(n^2)$ на FFW и $O(n^3)$ о механизме внимания, который генерирует n жетоны!

Как этого избежать? Оказывается, вместо того чтобы каждый раз выполнять полный прямой проход, мы можем сохранять некоторые промежуточные активации после каждого прямого прохода, что позволит нам избежать повторной обработки предыдущих токенов. В частности, поскольку данный токен взаимодействует только с предыдущими токенами во время вычисления долевого произведения, мы можем просто записать проекции ключей и значений каждого токена в новую структуру данных, называемую **KV-кэшем**. После того как мы сохраним эти проекции ключей и значений для предыдущих токенов, последующие токены смогут просто вычислить их $q_i \cdot k_j$ продукты без проведения новых операций по обмену на более ранние токены. Удивительно!

Исходя из этого, умозаключение состоит из двух ключевых частей:

- **Предварительная обработка:** при наличии длинного запроса мы одновременно обрабатываем все токены в запросе и сохраняем полученные активации (в частности, проекции «ключ-значение») в «кэше KV». Мы также сохраняем логиты для последнего токена.
- **Генерация:** имея кэш KV и предыдущие логиты, мы постепенно выбираем один токен из логитов, передаем его обратно в Transformer и получаем новый набор логитов для следующего шага. Мы также добавляем активации KV для этого нового токена в кэш KV. Мы повторяем этот процесс до тех пор, пока не встретим специальный токен `<EOS>` или не достигнем максимального ограничения по длине.

Вот схема выборки с использованием KV-кэша:

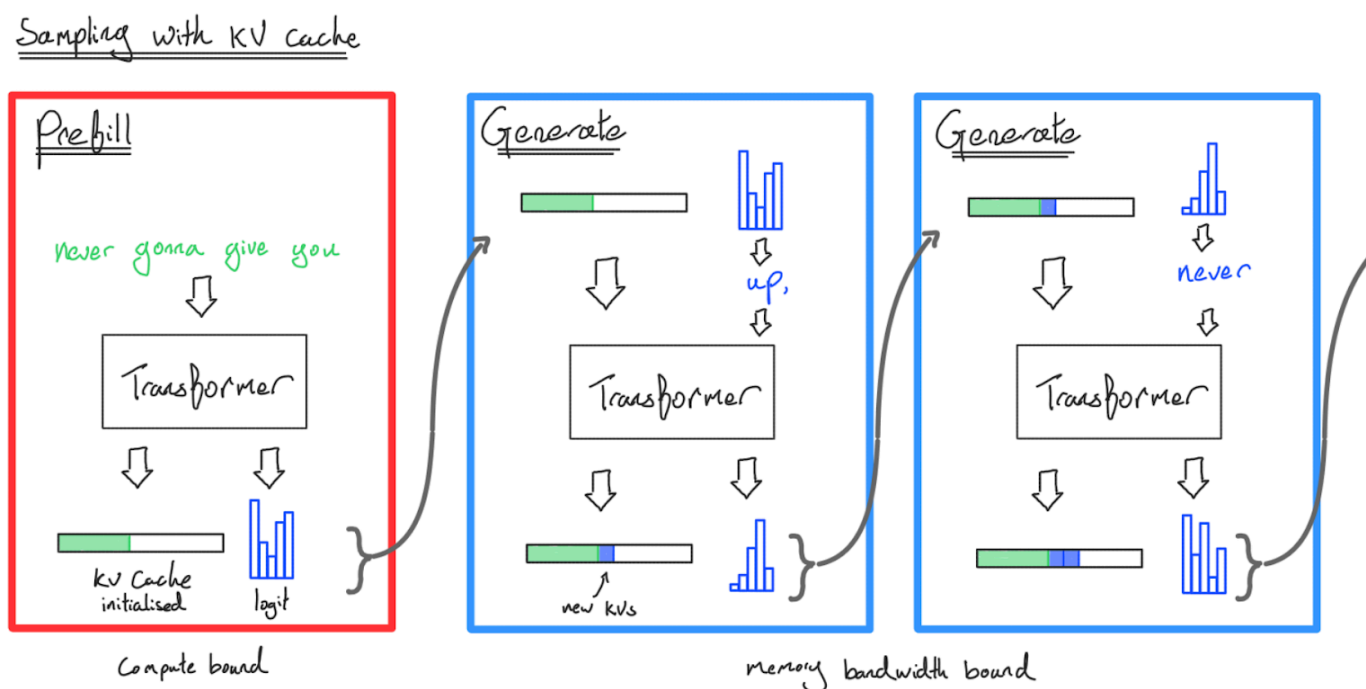


Рисунок: схема эффективной выборки в Transformer с использованием KV-кэша. **Prefill** обрабатывает нашу подсказку и сохраняет все активации «ключ-значение» для каждого токена в кэше. **Generation** берет этот кэш (и логиты последнего токена), выбирает новый токен и пропускает его через модель, используя KV-кэш и сохраняя проекции «ключ-значение» нового токена обратно в кэш. Это $O(n)$ алгоритм в блоке MLP.

Благодаря выборке с помощью KV-кэша мы снизили временную сложность генерации n токенов для $O(n)$ на FFW и $O(n^2)$ на внимание, поскольку мы никогда не перерабатываем предыдущий токен. Однако для генерации последовательности все равно требуется много прямых проходов — именно это происходит, когда вы делаете запрос к Gemini или ChatGPT и получаете ответ. Каждый токен — это (как правило) отдельный (но частично кэшированный) вызов Transformer для большой модели.

Вскоре мы увидим, что **предварительное заполнение** и **генерация** — это совершенно разные вещи. Вычисления в трансформерах — это две задачи под одной! По сравнению с обучением, KV-кэш также является новым и весьма сложным элементом.

Что именно мы хотим оптимизировать?

Прежде чем двигаться дальше, стоит обратить внимание на один совершенно новый аспект логического вывода — задержку. Если во время обучения нас интересует только пропускная способность (общее количество токенов, обработанных в секунду **на одном чипе**), то при логическом выводе нам нужно учитывать скорость генерации токенов (как **время до первого токена (TTFT)**, так и **задержку на один токен**). Например:

- **Автономный пакетный логический вывод** для оценки и генерации данных учитывает только массовую стоимость логического вывода и не обращает внимания на задержку при обработке отдельных выборок.
- **Чат-интерфейсы/задачи потоковой передачи** должны работать с минимальными затратами при масштабировании, иметь низкую задержку и генерировать токены достаточно быстро, чтобы превышать скорость чтения человеком.
- **Локальный логический вывод** (например, `llama.cpp` на вашем ноутбуке) должен обслуживать только одного пользователя за раз с минимально возможной задержкой, что может быть сопряжено с серьезными аппаратными ограничениями.

Максимальное использование аппаратного обеспечения по-прежнему имеет решающее значение и помогает снизить затраты и время до первого результата, но, в отличие от обучения, оно не *обязательно* приводит к улучшению пользовательского опыта во всех контекстах. Многие оптимизации на уровне ускорителей, систем и архитектуры моделей предполагают компромисс между задержкой, пропускной способностью, длиной контекста и даже качеством модели.

Более детальное представление Transformer

До сих пор мы рассматривали Transformer в основном как набор блоков прямого распространения. Хотя такой подход часто оправдан с точки зрения количества операций с плавающей запятой и объема памяти, его недостаточно для корректного моделирования логического вывода.² Как мы видели в [части 4](#), основными компонентами прямого прохода в трансформере являются:

1. **Набор линейных операций**, включая MLP (W_{in}, W_{out}) и внимание к прогнозам QKV и выходным прогнозам (W_Q, W_K, W_V , и W_O). Все они включают в себя считывание параметров и пакета активаций из HBM, выполнение нескольких операций с плавающей запятой и запись результата обратно в HBM.
2. **Внимание на основе скалярного произведения**. Нам нужно считать пакет проекций «ключ-значение» и пакет активаций запроса из HBM, выполнить несколько операций внутреннего произведения и операций softmax, а затем записать результат внимания обратно в HBM.
3. **Все остальное**, включая применение слоевых норм, функций активации, выборку токенов, обновление кэшей «ключ-значение» и позиционных эмбеддингов. Для них тоже требуется некоторое количество операций с плавающей запятой, но они либо доминируют, либо сливаются с вышеупомянутыми.

В следующих двух разделах мы рассмотрим каждый из этих аспектов в контексте предварительного заполнения и генерации и выясним, что может стать узким местом в нашей производительности. Что ограничивает нашу производительность — вычисления или память? Мы хотим подчеркнуть, насколько разными будут ответы в случае предварительного заполнения и генерации.

Линейные операции: что является узким местом?

Все наши линейные операции концептуально одинаковы, независимо от того, выполняются ли они в блоке MLP или в блоке внимания. Их вычислительная сложность зависит от размера пакета. Мы уже проводили эти вычисления в [разделе 1](#), но стоит повторить. Давайте рассмотрим однократное матричное умножение $\text{bf16}[B, D]$ партия, составленная $\text{bf16}[D, F]$ матрица. Это может быть большой блок MLP (W_{in} или W_{out}) или один из малых фокусов внимания (W_Q, W_K, W_V, W_O). Чтобы выполнить эту матричную операцию, нам нужно загрузить оба этих массива из HBM в MXU, выполнить умножение, а затем записать результат обратно в HBM. Как и раньше, у нас есть:

$$T_{\text{math}} = \frac{\text{Computation FLOPs}}{\text{Accelerator FLOPs/s}} = \frac{2BDF}{\text{Accelerator FLOPs/s}}$$

$$T_{\text{comms}} = \frac{\text{Communication Bytes}}{\text{Bandwidth Bytes/s}} = \frac{2BD + 2FD + 2BF}{\text{Bandwidth Bytes/s}}$$

TPU или графический процессор могут совмещать эти процессы, загружая данные в процессе вычислений, поэтому для ограничения вычислительных ресурсов нам нужно $T_{\text{math}} \geq T_{\text{comms}}$, или:

$$\frac{2BDF}{2BD + 2DF + 2BF} \geq \frac{\text{Accelerator FLOPs/s}}{\text{Bandwidth Bytes/s}} \stackrel{\text{TPU v5e}}{=} \frac{1.97E + 14}{8.20E + 11} = 240$$

где RHS — арифметическая интенсивность нашего оборудования. Теперь предположим, что D и F очень велики по сравнению с (обычно наши партии составляют не более 500 и D и $F > 10k$), мы можем упростить знаменатель, воспользовавшись тем фактом, что $2BD + 2DF + 2BF \approx 2DF$ что дает нам

$$\begin{aligned} \frac{2BDF}{2BD + 2DF + 2BF} &\approx \frac{2BDF}{2DF} \geq \frac{\text{Accelerator FLOPs/s}}{\text{Bandwidth Bytes/s}} \\ &\stackrel{\text{TPU v5e}}{=} \frac{1.97E + 14}{8.20E + 11} \implies B \geq 240 = B_{\text{crit}} \end{aligned}$$

Если мы квантуем веса или используем операции с меньшей точностью для матричного умножения, этот критический размер пакета может измениться. Например, если мы квантуем веса до int8 или fp8, B_{crit} уменьшается в 2 раза. Если мы выполняем операции над числами с плавающей запятой в формате int8 или fp8, B_{crit} увеличивается в 2 раза. Таким образом, если мы позволим $\beta = \text{битов на параметр/битов на активацию}$ и $\alpha_{\text{hbm}} = C/W_{\text{hbm}}$ на самом деле наш критический размер партии составляет $B_{\text{crit}} = \beta \alpha_{\text{hbm}}$.

Вывод: матричные умножители Transformer ограничены вычислительными ресурсами *тогда и только тогда*, когда размер пакета **токенов** на реплику больше, чем $B_{\text{crit}} = C/W_{\text{hbm}} \cdot (\text{bits per param/bits per activation}) = \beta \cdot \alpha_{\text{hbm}}$. Для активации bf16 на TPU v5e требуется 240 токенов. Для H100 — около 280 токенов.

Во время обучения мы будем сталкиваться с высокой интенсивностью вычислений при всех матричных умножениях, поскольку используем одни и те же веса для очень большой выборки. **Такая высокая арифметическая интенсивность сохраняется и при предварительном заполнении, поскольку пользовательские запросы обычно состоят из сотен, если не тысяч, токенов.** Как мы уже видели, аппаратная арифметическая интенсивность TPUv5e составляет 240, поэтому, если последовательность длиной более 240 токенов поступает в плотную модель, работающую на этом оборудовании с bf16, мы можем ожидать, что вычислительные ресурсы будут ограничены и все будет в порядке. Подсказки короче этой технически можно объединять для более эффективного использования, но обычно в этом нет необходимости.

Вывод: во время предварительного заполнения все матричные умножения практически всегда зависят от вычислительных ресурсов. Таким образом, для максимизации пропускной способности на чип (и снижения стоимости) и сокращения времени до первого выполнения (TTFT) достаточно просто добиться максимальной загрузки оборудования или MFU (утилизации модельных операций с плавающей запятой). Если запросы не очень короткие, пакетная обработка на уровне каждого запроса лишь увеличивает задержку при незначительном улучшении пропускной способности предварительного заполнения.

Однако во время генерации для каждого запроса мы можем выполнять прямые проходы только по одному токenu за раз, поскольку между этапами существует последовательная зависимость! Таким образом, мы можем (легко) добиться хорошей загрузки, объединяя несколько запросов в пакеты и распараллеливая их по пакетному измерению. Мы поговорим об этом подробнее позже, но на самом деле объединить множество параллельных запросов в пакеты без ущерба для задержки сложно. По этой причине **намного труднее задействовать аппаратные вычисления с плавающей запятой при генерации.**

Вывод: при генерации общий размер пакета токенов должен быть больше B_{crit} . Ограничение по вычислениям для линейных операций и операций прямого распространения (240 для параметров bf16 на TPU v5e). Поскольку генерация происходит последовательно, токен за токеном, нам приходится объединять несколько запросов в один, а это непросто!

Стоит отметить, насколько это масштабно! Размер пакета в 240 означает, что одновременно генерируется 240 параллельных запросов и 240 отдельных KV-кэшей для плотных моделей. Это означает, что на практике добиться такого сложно, за исключением некоторых режимов массового логического вывода. В то же время протолкнуть более 240 токенов во время предварительного заполнения довольно просто, хотя при увеличении разреженности данных следует соблюдать осторожность.

Обратите внимание, что точное число зависит от типа квантования и аппаратного обеспечения. Ускорители часто могут выполнять больше арифметических операций с меньшей точностью. Например, если у нас есть параметры типа int8, но вычисления выполняются в bf16, критический размер пакета снижается до 120. При активации int8 и параметрах int8 он снова возрастает до 240, поскольку TPUv5e может выполнять 400 операций в секунду с типом int8 x int8.

А что насчет внимания?

Все становится еще сложнее, когда мы рассматриваем операцию внимания с точечным произведением, особенно с учетом кэшей KV. Давайте рассмотрим только одну голову внимания с чистым многоголовым вниманием. В рамках одного слияния Flash Attention мы³:

1. Прочтите Q активации формы $\text{bf16}[B, T, D]$ из HBM.
2. Прочитайте кэш, представляющий собой пару $\text{bf16}[B, S, D]$ тензоры из HBM.
3. Выполнить $2BSTD$ Проваливается в QK matmul. Благодаря Flash Attention нам не нужно писать $\text{bf16}[B, S, T]$ Матрица внимания возвращается в HBM.
4. Выполнить $2BSTD$ во внимании AV matmul.
5. Запишите результат $\text{bf16}[B, T, D]$ Преобразуем тензор обратно в HBM.

Сложив все это вместе, мы получаем:

$$\text{Multiheaded Attention Arithmetic Intensity} = \frac{4BSTD}{4BSD + 4BTD} = \frac{ST}{S + T}$$

Для предварительной заливки, $S = T$ Поскольку мы занимаемся самоконтролем, все упрощается до $T^2/2T = T/2$ Это здорово, потому что означает, что **арифметическая интенсивность внимания во время предварительного заполнения** $\Theta(T)$ Это означает, что вычислительная сложность внимания довольно высока. Пока длина нашей последовательности достаточно велика, все будет в порядке!

Но поскольку у поколения тривиальная последовательность dim , то T и D Поскольку dims сокращаются, мы можем сделать следующее приближение:

$$S \gg T = 1 \implies \frac{ST}{S + T} \approx 1$$

This is bad, since it means we cannot do anything to improve the arithmetic intensity of attention during generation. We're doing a tiny amount of FLOPs while loading a massive KV cache. **So we're basically always memory bandwidth-bound during attention!**

Takeaway: during prefill, attention is usually compute bound for any reasonable sequence length (roughly > 480 tokens) while during generation our arithmetic intensity is low and constant, so we are always memory bandwidth-bound.

Why is this, conceptually? Mainly, we're compute-bound in linear portions of the model because the parameters (the memory bandwidth-heavy components) are reused for many batch items. However, every batch item has its own KV cache, so a bigger batch size means more KV caches. We will almost *always* be memory bound here unless the architecture is adjusted aggressively.

This also means you will get diminishing returns on throughput from increasing batch size once params memory becomes comparable to KV cache memory. The degree to which the diminishing returns hurt you depends on the ratio of parameter to KV cache bytes for a single sequence, i.e. roughly the ratio $2DF/SHK$. Since $HK \approx D$, this roughly depends on the ratio of F to C , the sequence length. This also depends on architectural modifications that make the KV cache smaller (we'll say more in a moment).

Theoretical estimates for LLM latency and throughput

From this math, we can get pretty good bounds on the step time we should aim for when optimizing. (**Note: if there is one thing we want the reader to take away from this entire chapter, it's the following**). For small batch sizes during generation (which is common), we can lower-bound our per-step latency by assuming we're memory bandwidth bound in both the attention and MLP blocks:

$$\text{Theoretical Min Step Time} = \frac{\text{Batch Size} \times \text{KV Cache Size} + \text{Parameter Size}}{\text{Total Memory Bandwidth}}$$

Similarly, for throughput:

$$\text{Theoretical Max Tokens/s} = \frac{\text{Batch Size} \times \text{Total Memory Bandwidth}}{\text{Batch Size} \times \text{KV Cache Size} + \text{Parameter Size}}$$

Eventually, as our batch size grows, FLOPs begin to dominate parameter loading, so in practice we have the more general equation:

$$\text{Theoretical Step Time (General)} = \frac{\text{Batch Size} \times \text{KV Cache Size}}{\text{Total Memory Bandwidth}} + \max\left(\frac{2 \times \text{Batch Size} \times \text{Parameter Count}}{\text{Total FLOPs/s}}, \frac{\text{Parameter Size}}{\text{Total Memory Bandwidth}}\right) \quad (1)$$

Attention (always bandwidth-bound) MLP (can be compute-bound)

where the attention component (left) is never compute-bound, and thus doesn't need a FLOPs roofline. These are fairly useful for back-of-the-envelope calculations, e.g.

Pop Quiz: Assume we want to take a generate step with a batch size of 4 tokens from a 30B parameter dense model on TPU v5e 4x4 slice in int8 with bf16 FLOPs, 8192 context and 100 kB / token KV caches. What is a reasonable lower bound on the latency of this operation? What if we wanted to sample a batch of 256 tokens?

► [Click here for the answer.](#)

As you can see, there's a clear tradeoff between throughput and latency here. Small batches are fast but don't utilize the hardware well. Big batches are slow but efficient. Here's the latency-throughput Pareto frontier calculated for some older PaLM models (from the [ESTL paper](#) [1]):

Decoding Latency vs. Cost

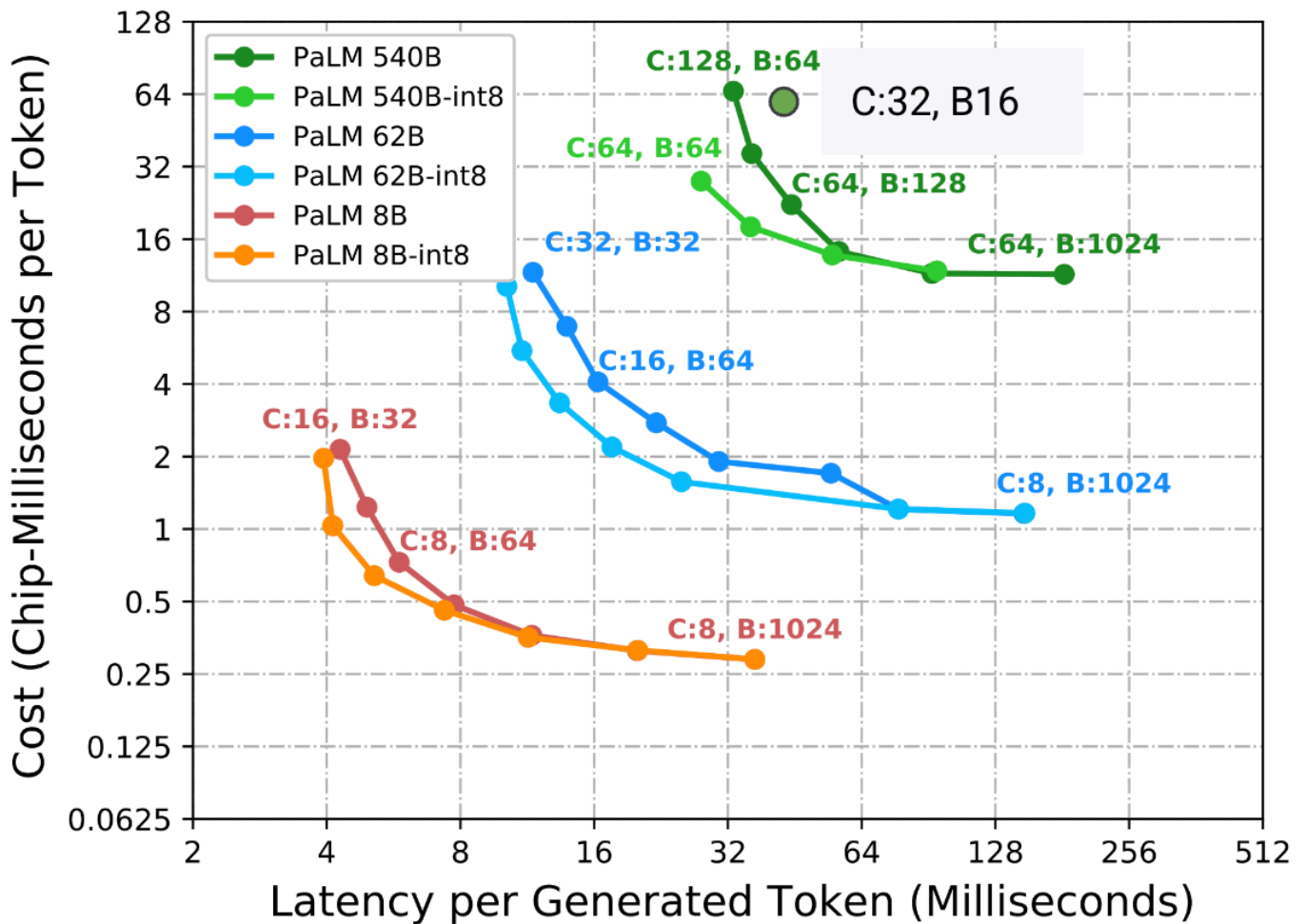


Figure: Pareto frontier of cost (read: throughput) versus latency for several PaLM models. Note how chip count (C) and batch size (B) moves you along the Pareto frontier, with the exception of the green dot (C:32 B:16 for PaLM 540B) where the available memory prevented the setup from supporting a good batch size and caused throughput to suffer. Note how throughput generally tends to flatten around after the batch size 240. int8 weights offers a better latency-throughput pareto optimal, but not a better max throughput.

Not only do we trade off latency and throughput with batch size as knob, we may also prefer a larger topology to a smaller one so we can fit larger batches if we find ourselves limited by HBM. The [next section](#) explores this in more detail.

Takeaway: if you care about generation throughput, use the largest per-chip batch size possible. Any per-chip batch size above the TPU arithmetic intensity (B_{crit} , usually 120 or 240) will maximize throughput. You may need to increase your topology to achieve this. Smaller batch sizes will allow you to improve latency at the cost of throughput.

► There are some caveats to this from a hardware standpoint. [Click here for some nits.](#)

What about memory?

We've spent some time looking at bandwidth and FLOPs, but not at memory. The memory picture looks a lot different at inference time, thanks to our new data structure, the KV cache. For this section, let's pick a real model (LLaMA 2-13B) to demonstrate how different things look:

hyperparam	value
L (num_layers)	40
D (d_model)	5,120
F (ffw_dimension)	13,824
N (num_heads)	40
K (num_kv_heads)	40
H (qkv_dim)	128
V (num_embeddings)	32,000

What’s using memory during inference? Well, obviously, our parameters. Counting those, we have:

param	formula	size (in bytes)
FFW params	$d_model^2 \times ffw_multiplier \times 3$ (for SwiGLU gate, up, and down projections) $\times n_layers$	$5,120 \times 5,120 \times 2.7 \times 3 \times 40 = \mathbf{8.5e9}$
Vocab params	2 (input and output embeddings) $\times n_embeddings \times d_model$	$2 \times 32,000 \times 5,120 = \mathbf{0.3e9}$
Attention params	$[2 \text{ (} q \text{ and output)} \times d_model \times n_heads \times d_qkv + 2 \text{ (for } k \text{ and } v) \times d_model \times n_kv_heads \times d_qkv] \times n_layers$	$(2 \times 5,120 \times 40 \times 128 + 2 \times 5,120 \times 40 \times 128) \times 40 = \mathbf{4.2e9}$

Adding these parameters up, we get $8.5e9 + 4.2e9 + 0.3e9 = \mathbf{13e9}$ total parameters, just as expected. As we saw in the previous sections, during training we might store our parameters in bfloat16 with an optimizer state in float32. That may use around 100GB of memory. That pales in comparison to our gradient checkpoints, which can use several TBs.

How is inference different? During inference, we store one copy of our parameters, let’s say in bfloat16. That uses 26GB — and in practice we can often do much better than this with quantization. There’s no optimizer state or gradients to keep track of. Because we don’t checkpoint (keep activations around for the backwards pass), our activation footprint is negligible for both prefill⁴ and generate. If we prefill 8k tokens, a single activation only uses around $\mathbf{8,192 \times 5,120 \times 2 \text{ bytes} = 80MB}$ of memory. Longer prefills can be broken down into many smaller forward passes, so it’s not a problem for longer contexts either. Generation uses even fewer tokens than that, so activations are negligible.

The main difference is the KV cache. These are the keys and value projections for all past tokens, bounded in size only by the maximum allowed sequence length. The total size for T tokens is

$$\text{KV cache size} = 2 \cdot \text{bytes per float} \cdot H \cdot K \cdot L \cdot T$$

where H is the dimension of each head, K is the number of KV heads, L is the number of layers, and the 2 comes from storing both the keys and values.

This can get big very quickly, even with modest batch size and context lengths. For LLaMA-13B, a KV cache for a single 8192 sequence at bf16 is

$$8192 \text{ (} T \text{)} \times 40 \text{ (} K \text{)} \times 128 \text{ (} H \text{)} \times 40 \text{ (} L \text{)} \times 2 \text{ (bytes)} \times 2 = \mathbf{6.7GB}$$

Just 4 of these exceed the memory usage of our parameters! To be clear, LLaMA 2 was not optimized for KV cache size at longer contexts (it isn’t always this bad, since usually H is much smaller, as in LLaMA-3), but this is still illustrative. We cannot neglect these in memory or latency estimates.

Modeling throughput and latency for LLaMA 2-13B

Let’s see what happens if we try to perform generation perfectly efficiently at different batch sizes on 8xTPU v5es, up to the critical batch size (240) derived earlier for maximum theoretical throughput.

Batch Size	1	8	16	32	64	240
KV Cache Memory (GiB)	6.7	53.6	107.2	214.4	428.8	1608
Total Memory (GiB)	32.7	79.6	133.2	240.4	454.8	1634
Theoretical Step Time (ms)	4.98	12.13	20.30	36.65	69.33	249.09
Theoretical Throughput (tokens/s)	200.61	659.30	787.99	873.21	923.13	963.53

8x TPU v5es gives us 128GiB of HBM, 6.5TiB/s of HBM bandwidth (0.82TiB/s each) and 1600TF/s of compute.

For this model, increasing the batch size does give us better throughput, but we suffer rapidly diminishing returns. We OOM beyond batch size 16, and need an order of magnitude more memory to go near 240. A bigger topology can improve the latency, but we’ve hit a wall on the per chip throughput.

Let’s say we keep the total number of params the same, but magically make the KV cache 5x smaller (say, with 1:5 [GMOA](#), which means we have 8 KV heads shared over the 40 Q heads — see next section for more details).

Batch Size	1	8	16	32	64	240
KV Cache Memory (GiB)	1.34	10.72	21.44	42.88	85.76	321.6
Total Memory (GiB)	27.34	36.72	47.44	68.88	111.76	347.6
Theoretical Step Time (ms)	4.17	5.60	7.23	10.50	17.04	52.99
Theoretical Throughput (tokens/s)	239.94	1,429.19	2,212.48	3,047.62	3,756.62	4,529.34

With a smaller KV cache, we still have diminishing returns, but the theoretical throughput per chip continues to scale up to batch size 240. We can fit a much bigger batch of 64, and latency is also consistently better at all batch sizes. The latency, maximum throughput, and maximum batch size all improve dramatically! In fact, later LLaMA generations used this exact optimization — LLaMA-3 8B has 32 query heads and 8 KV heads ([source](#)).

Takeaway: In addition to params, the size of KV cache has a lot of bearing over the ultimate inference performance of the model. We want to keep it under control with a combination of architectural decisions and runtime optimizations.

Tricks for Improving Generation Throughput and Latency

Since the original [Attention is All You Need paper](#), many techniques have been developed to make the model more efficient, often targeting the KV cache specifically. Generally speaking, a smaller KV cache makes it easier to increase batch size and context length of the generation step without hurting latency, and makes life easier for the systems surrounding the Transformer (like request caching). Ignoring effects on quality, we may see:

Grouped multi-query attention (aka GQA, MQA): We can reduce the number of KV heads, and share them with many Q heads in the attention mechanism. In the extreme case, it is possible to share a single KV head across all Q heads. This reduces the KV cache by a factor of the Q:KV ratio over pure MHA, and it has been observed that the performance of models is relatively insensitive to this change.

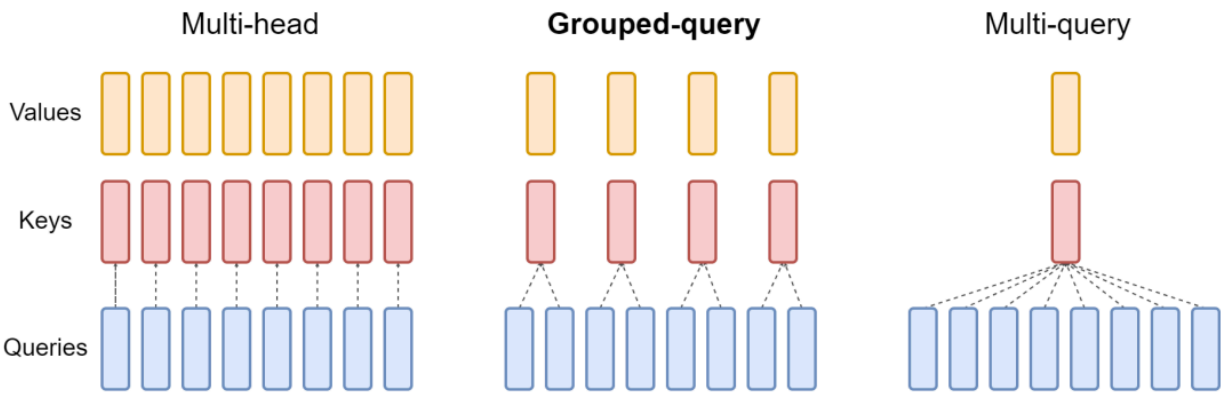


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

This also effectively increases the arithmetic intensity of the attention computation (see Question 4 in [Section 4](#)).

Mixing in some local attention layers: Local attention caps the context to a small to moderately sized max length. At training time and prefill time, this involves masking the attention matrix to a diagonal strip instead of a triangle. This effectively caps the size of the max length of the KV cache for the local layers. By mixing in some local layers into the model with some global layers, the KV cache is greatly reduced in size at contexts longer than the local window.

Sharing KVs across layers: The model can learn to share the same KV caches across layers in some pattern. Whilst this does reduce the KV cache size, and provide benefits in increasing batch size, caching, offline storage, etc., shared KV caches may need to be read from HBM multiple times, *so it does not necessarily improve the step time*.

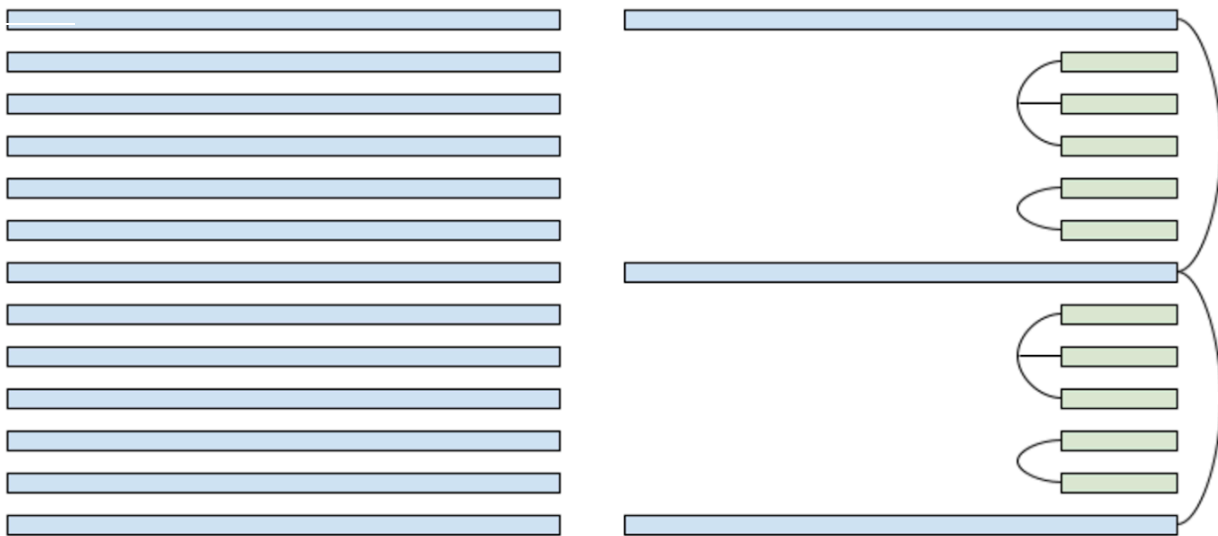


Figure 1. Left: Standard transformer design where every attention is global attention. Right: The attention design in our production model. Blue boxes indicate global attention, green boxes indicate local attention, and curves indicate KV-sharing. For global attention layers, we share KV across multiple non-adjacent layers. This illustration depicts only a subset of the layers in the full model.

Left: Multiple layers of pure global attention. **Right:** An example of some global/local interleaving pattern with sharing with adjacent layers. Source: [Character.ai blog](#).

Quantization: Inference is usually less sensitive to the precision of parameters and KV's. By quantizing the parameters and KV cache (e.g. to int8, int4, **fp8** etc.), we can save on memory bandwidth on both, decrease the batch size required to reach the compute roofline and save memory to run at bigger batch sizes. Quantization has the added advantage that even if the model was not trained with quantization it can often be applied post training.

Using ragged HBM reads and Paged Attention: We allocated 8k of context for each KV cache in the calculations above but it is often not necessary to read the entire KV cache from memory — requests have a wide range of length distributions and don't use the max context of the model, so we can often implement kernels (e.g. Flash Attention variants) that only read the non-padding part of the KV cache.

Paged Attention [2] is a refinement upon this that stores KV caches in OS-style page tables and mostly avoids padding the KV caches altogether. This adds a lot of complexity but means every batch only uses as much memory as it needs. This is a runtime optimization, so again it is indifferent to architecture.

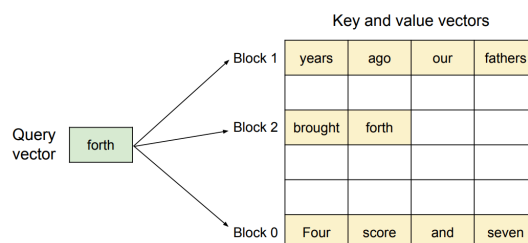


Figure 5. Illustration of the PagedAttention algorithm, where the attention key and values vectors are stored as non-contiguous blocks in the memory.

Figure: during generation, a single token ("forth") attends to multiple KV cache blocks/pages. By paging the KV cache, we avoid loading or storing more memory than we need to. Taken from the [PagedAttention paper](#).

Big Picture: All told, these KV cache optimizations can reduce KV cache sizes by over an order of magnitude compared to a standard MHA Transformer. This can lead to an order-of-magnitude improvement in the overall cost of the Transformer.

Distributing Inference Over Multiple Accelerators

So far we've handwaved how we're scaling beyond a single chip. Following [Section 5](#), let's explore the different strategies available to us and their tradeoffs. As always, we will look at prefill and generation separately.

Prefill

From a roofline standpoint, **prefill is almost identical to training** and almost all the same techniques and tradeoffs apply — model (Megatron) parallelism, sequence sharding (for sufficiently long context), pipelining, even FSDP are all viable! You just have to keep the KV's kicking around so you can do generation later. As in training, increasing the number of chips gives us access to more FLOPs/s (for potentially lower TTFT), but adds communication overhead (potentially reducing throughput per

chip).

The general rule for sharding prefill: here's a general set of rules for prefill. We'll assume we're doing prefill on a single sequence only (no batch dimension):

1. *Model sharding:* We typically do some amount of model parallelism first, up to the point we become ICI-bound. As we saw in [Section 5](#), this is around $F/2200$ for 1 axis (usually around 4-8 way sharding).
2. *Sequence parallelism:* Beyond this, we do sequence parallelism (like data parallelism but sharding across the sequence dimension). While sequence parallelism introduces some extra communication in attention, it is typically fairly small at longer contexts. As with training, we can overlap the communication and computation (using collective matmuls for Megatron and ring attention respectively).

Takeaway: during prefill, almost any sharding that can work during training can work fine. Do model parallelism up to the ICI bound, then do sequence parallelism.

Generation

Generation is a more complicated beast than prefill. For one thing, it is harder to get a large batch size because we need to batch many requests together. Latency targets are lower. Together, these mean we are typically more memory-bound and more sensitive to communication overhead, which restrict our sharding strategies:

1. **FSDP is impossible:** since we are memory-bound in loading our parameters and KV caches from HBM to the MXU, we do not want to move them via ICI which is orders of magnitudes slower than HBM. *We want to move activations rather than weights.* This means methods similar to FSDP are usually completely unviable for generation.⁵
2. **There is no reason to do data parallelism:** pure data parallelism is unhelpful because it replicates our parameters and doesn't help us load parameters faster. You're better off spinning up multiple copies of the model instead.⁶
3. **No sequence = no sequence sharding.** Good luck sequence sharding.

This mostly leaves us with variants of model sharding for dense model generation. As with prefill, the simplest thing we can do is simple model parallelism (with activations fully replicated, weights fully sharded over hidden dimension for the MLP) up to 4-8 ways when we become ICI bound. However, since we are often memory bandwidth bound, we can actually go beyond this limit to improve latency!

Note on ICI bounds for generation: during training we want to be compute-bound, so our rooflines look at when our ICI comms take longer than our FLOPs. However, during generation, if we're memory bandwidth bound by parameter loading, we can increase model sharding beyond this point and improve latency at a minimal throughput cost (in terms of tokens/sec/chip). More model sharding gives us more HBM to load our weights over, and our FLOPs don't matter.⁷ Let's look at how much model parallelism we can do before it becomes the bottleneck.

$$T_{\text{HBM comms}} = \frac{2DF}{Y \cdot W_{\text{hbm}}} \quad T_{\text{ICI comms}} = \frac{2BD}{W_{\text{ici}}} \\ T_{\text{ICI comms}} > T_{\text{HBM comms}} \rightarrow \frac{W_{\text{hbm}}}{W_{\text{ici}}} > \frac{F}{Y \cdot B} \rightarrow Y > F/(B \cdot \beta)$$

where $\beta = W_{\text{hbm}}/W_{\text{ici}}$. This number is usually around 8 for TPU v5e and TPU v6e. That means e.g. if F is 16,384 and B is 32, we can in theory do model parallelism up to $16384 / (32 * 8) = 64$ ways without a meaningful hit in throughput. This assumes we can fully shard our KV caches 64-ways which is difficult: we discuss this below.

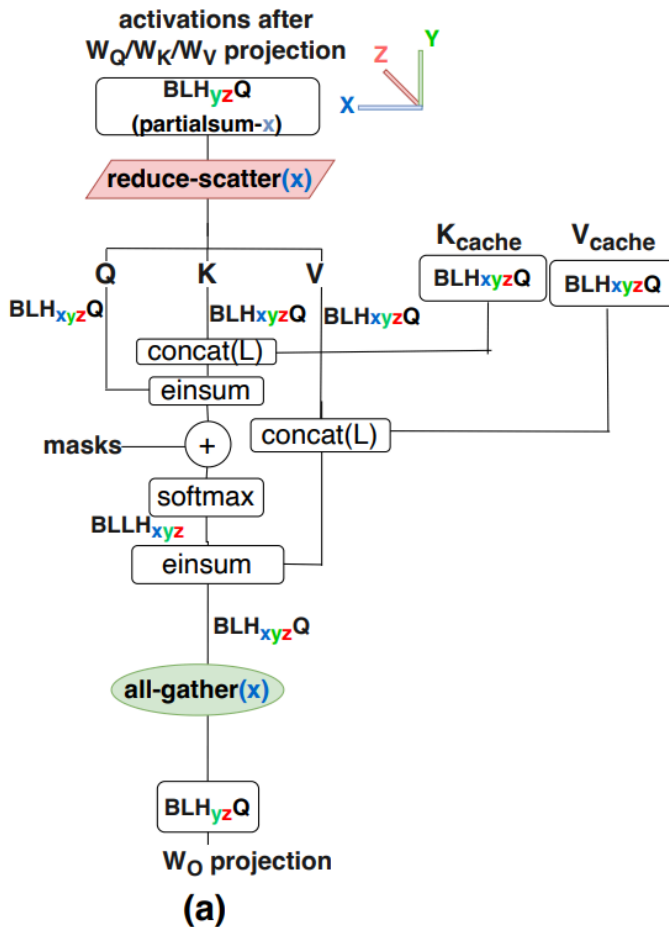
For the attention layer, we also model shard attention W_Q and W_O over heads Megatron style. The KV weights are quite small, and replicating them is often cheaper than sharding beyond -way sharding.

Takeaway: our only options during generation are variants of model parallelism. We aim to move activations instead of KV caches or parameters, which are larger. When our batch size is large, we do model parallelism up to the FLOPs-ICI bound (F/α). When our batch size is smaller, we can improve latency by model sharding more (at a modest throughput cost). When we want to model shard more ways than we have KV heads, we can shard our KVs along the batch dimension as well.

Sharding the KV cache

We also have an additional data structure that needs to be sharded — the KV cache. Again, we almost always prefer to avoid replicating the cache, since it is the primary source of attention latency. To do this, we first Megatron-shard the KVs along the head dimension. This is limited to -way sharding, so for models with a small number of heads, we shard the head dimension as much as possible and then shard along the batch dimension, i.e. $KV[2, B_Z, S, K_Y, H]$. This means the KV cache is completely distributed.

Multi-head attention (sharded over heads)



Multi-query attention (sharded over batch)

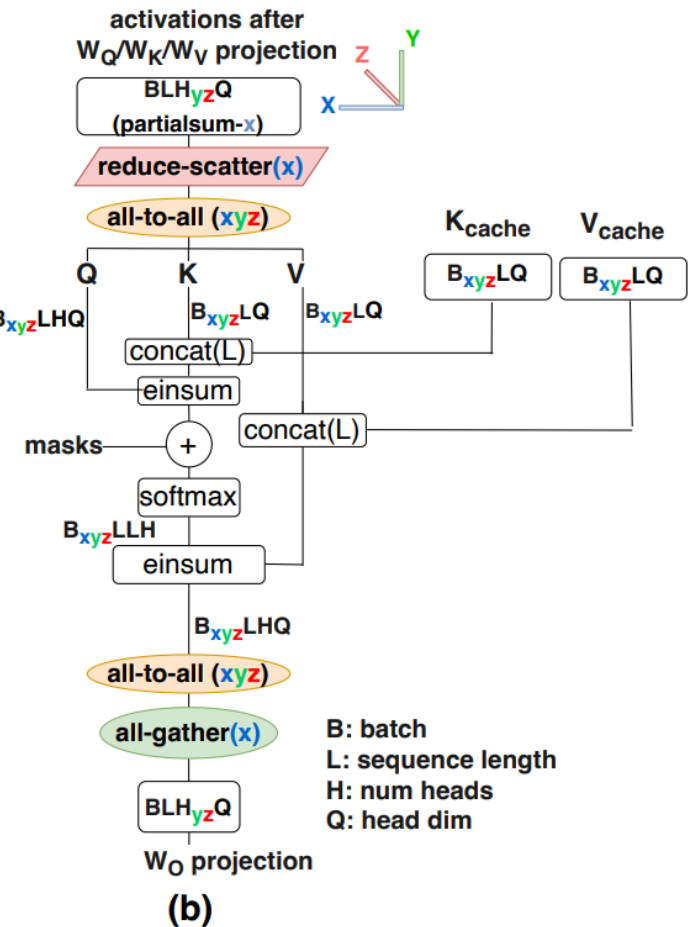


Figure: comparison of the attention mechanism with (a) Multi head attention with pure model sharding and (b) Multiquery attention with batch sharding of the KV cache. Notice how we need two extra AllToAlls to shift the activations from model sharding to batch sharding, so they can act on the KV caches.

The cost of this is two AllToAlls every attention layer — one to shift the Q activations to the batch sharding so we can compute attention with batch sharding, and one to shift the batch sharded attention output back to pure model sharded.

► [Here's the full algorithm!](#)

- **Sequence sharding:** If the batch size is too small, or the context is long, we can sequence shard the KV cache. Again, we pay a collective cost in accumulating the attention across shards here. First we need to AllGather the Q activations, and then accumulate the KVs in a similar fashion to Flash Attention.

Designing an Effective Inference Engine

So far we've looked at how to optimize and shard the individual prefill and generate operations efficiently in isolation. To actually use them effectively, we need to design an inference engine which can feed these two operations at a point of our choosing on the latency/throughput Pareto frontier.

The simplest method is simply to run a batch of prefill, then a batch of generations:

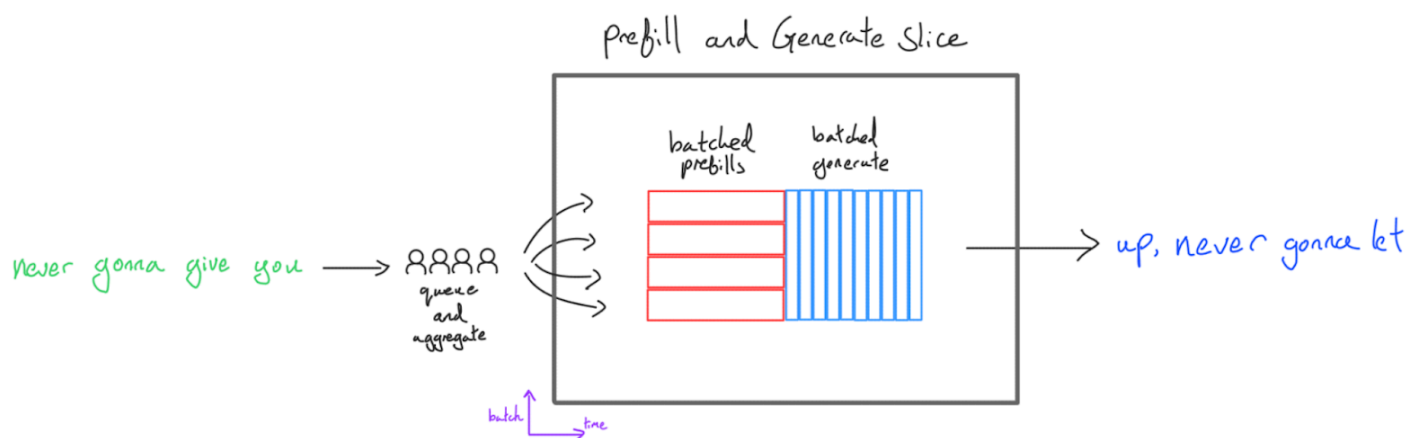


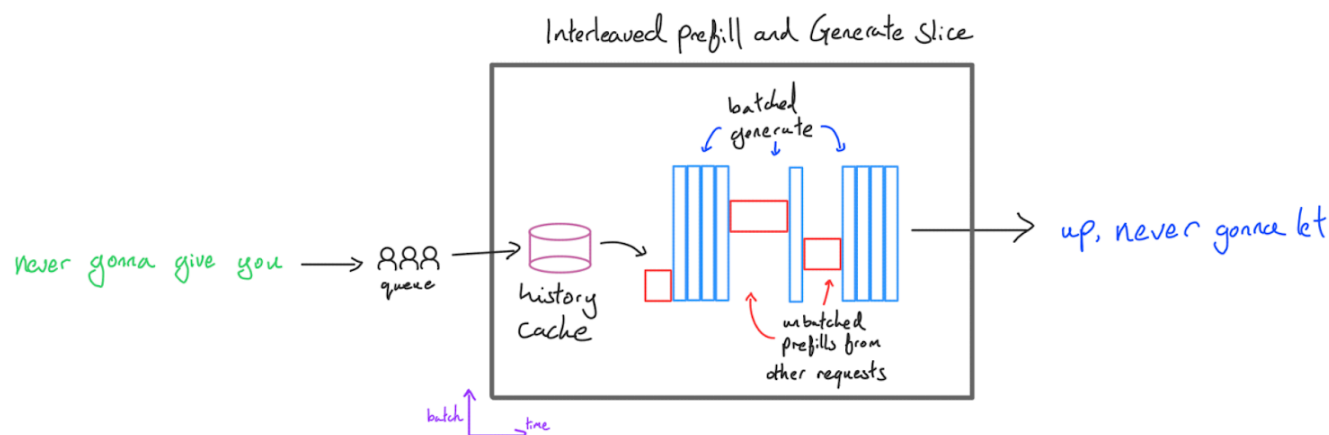
Figure: in the simplest setup, requests are aggregated, and the server alternates between running a batch of prefills and calling the generate function until completion for all sequences.

This is easy to implement and is the first inference setup in most codebases, but it has multiple drawbacks:

1. **Latency is terrible.** We couple the prefill and generate batch size. Time to first token (TTFT) is terrible at big prefill batch sizes — you need to finish all prefills before any users can see any tokens. Generate throughput is terrible at small batch sizes.
2. **We block shorter generations on longer ones.** Many sequences will finish before others, leaving empty batch slots during generation, hurting generate throughput further. The problem exacerbates as batch size and generation length increases.
3. **Prefills are padded.** Prefills are padded to the longest sequence and we waste a lot of compute. There are solutions for this, but historically XLA made it quite difficult to skip these FLOPs. Again this becomes worse the bigger the batch size and prefill sequence length.
4. **We're forced to share a sharding between prefill and generation.** Both prefill and generate live on the same slice, which means we use the same topology and shardings (unless you keep two copies of the weights) for both and is generally unhelpful for performance e.g. generate wants a lot more model sharding.

Therefore this method is only recommended for edge applications (which usually only cares about serving a single user and using hardware with less FLOPs/byte) and rapid iteration early in the lifecycle of a Transformer codebase (due to its simplicity).

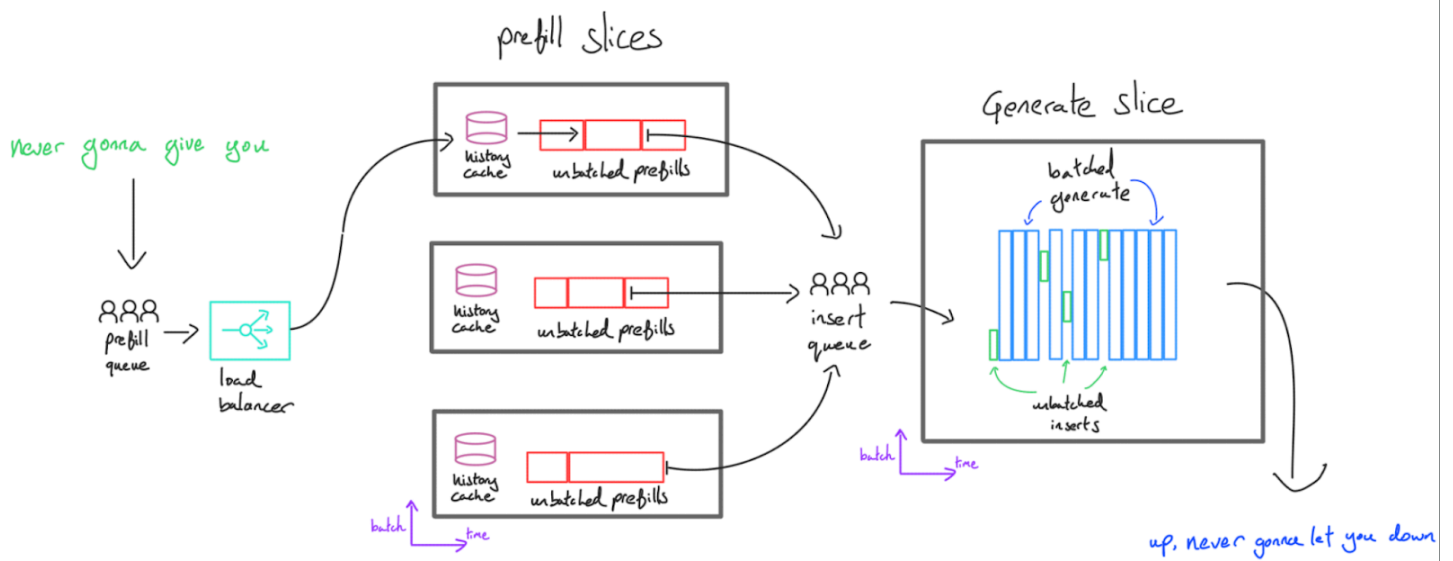
A slightly better approach involves performing prefill at batch size 1 (where it is compute-bound but has reasonable latency) but batch multiple requests together during generation:



This will avoid wasted TTFT from batched prefill while keeping generation throughput high. We call this an **interleaved** configuration, since we "interleave" prefill and generation steps. This is very powerful for bulk generation applications like evaluations where throughput is the main goal. The orchestrator can be configured to prioritise prefill the moment any generation slots open up, ensuring high utilisation even for very large generation batch sizes. We can also avoid padding our prefill to the maximum length, since it isn't batched with another request.

The main disadvantage is that when the server is performing a prefill, the generation of all other requests pauses since all the compute resources will be consumed by the prefill. User A whose response is busy decoding will be blocked by user B whose prefill is occurring. This means even though TTFT has improved, the token generation will be jittery and slow on average, which is not a good user experience for many applications — other user's prefills are on the critical path of the overall latency of a request.

To get around this, we separate decode and prefill. While Transformer inference can be done on one server, it is often better from a latency standpoint to execute the two different tasks on two sets of TPUs/GPUs. Prefill servers generate KV caches that get sent across the network to the generate servers, which batch multiple caches together and generate tokens for each of them. We call this **"disaggregated"** serving.



This provides a few advantages:

1. **Low latency at scale:** A user's request never blocks on another user's, except if there is insufficient prefill capacity. The request should be immediately prefilled, then sent to the generation server, then immediately slotted into the generation buffer. If we expect many concurrent requests to come in, we can scale the number of prefill servers independently from the number of generate servers so users are not left in the prefill queue for an extended period of time.
2. **Specialization:** Quite often, the latency-optimal parameter sharding strategy/hardware topology for prefill and generate is quite different (for instance, more model parallelism is useful for generate but not prefill). Constraining the two operations to use the same sharding hurts the performance of both, and having two sets of weights uses memory. Also, by moving prefill onto its own server, it doesn't need to hold any KV caches except the one it's currently processing. That means we have a lot more memory free for history caching (see the next section) or optimizing prefill latency.

One downside is that the KV cache now needs to be shifted across the network. This is typically acceptable but again provides a motivation for reducing KV cache size.

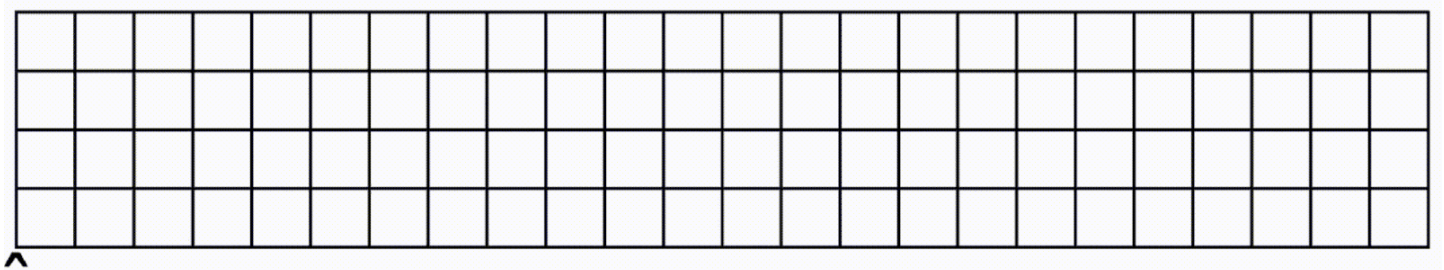
Takeaway: for latency-sensitive, high-throughput serving, we typically have to separate prefill and generation into separate servers, with prefill operating at batch 1 and generation batching many concurrent requests together.

Continuous batching

Problem (2) above motivates the concept of **continuous batching**. We optimize and compile:

- A prefill function that handles variable context lengths and inserts results into a KV buffer with some maximum batch size and context length/number of pages.
- A generate function which takes in the KV cache, and performs the generation step for all currently active requests.

We then combine these functions with an orchestrator which queues the incoming requests, calls prefill and generate depending on the available generate slots, handles history caching (see next section) and streams the tokens out.



Prefix caching

Since prefill is expensive and compute-bound (giving us less headroom), one of the best ways to reduce its cost is to do less of it. Because LLMs are autoregressive, the queries ["I", "like", "dogs"] and ["I", "like", "cats"] produce KV caches that are identical in the first two tokens. What this means is that, in principle, if we compute the "I like dogs" cache first and then the "I like cats" cache, we only need to do 1 / 3 of the compute. We can save most of the work by reusing the cache. This is particularly powerful in a few specific cases:

1. **Chatbots:** most chatbot conversations involve a back-and-forth dialog that strictly appends to itself. This means if we can save the KV caches from each dialog turn, we can skip computation for all but the newest tokens.
2. **Few-shot prompting:** if we have any kind of few-shot prompt, this can be saved and reused for free. System instructions often have this form as well.

The only reason this is hard to do is memory constraints. As we’ve seen, KV caches are big (often many GB), and for caching to be useful we need to keep them around until a follow-up query arrives. Typically, any unused HBM on the prefill servers can be used for a local caching system. Furthermore, accelerators usually have a lot of memory on their CPU hosts (e.g. a 8xTPUv5e server has 128GiB of HBM, but around 450GiB of Host DRAM). This memory is much slower than HBM — too slow to do generation steps usually — but is fast enough for a cache read. In practice:

- Because the KV cache is local to the set of TPUs that handled the initial request, we need some form of affinity routing to ensure follow-up queries arrive at the same replica. This can cause issues with load balancing.
- A smaller KV cache is helpful (again) — it enables us to save more KV caches in the same amount of space, and reduce read times.
- The KV cache and their lookups can be stored quite naturally in a tree or trie. Evictions can happen on an LRU basis.

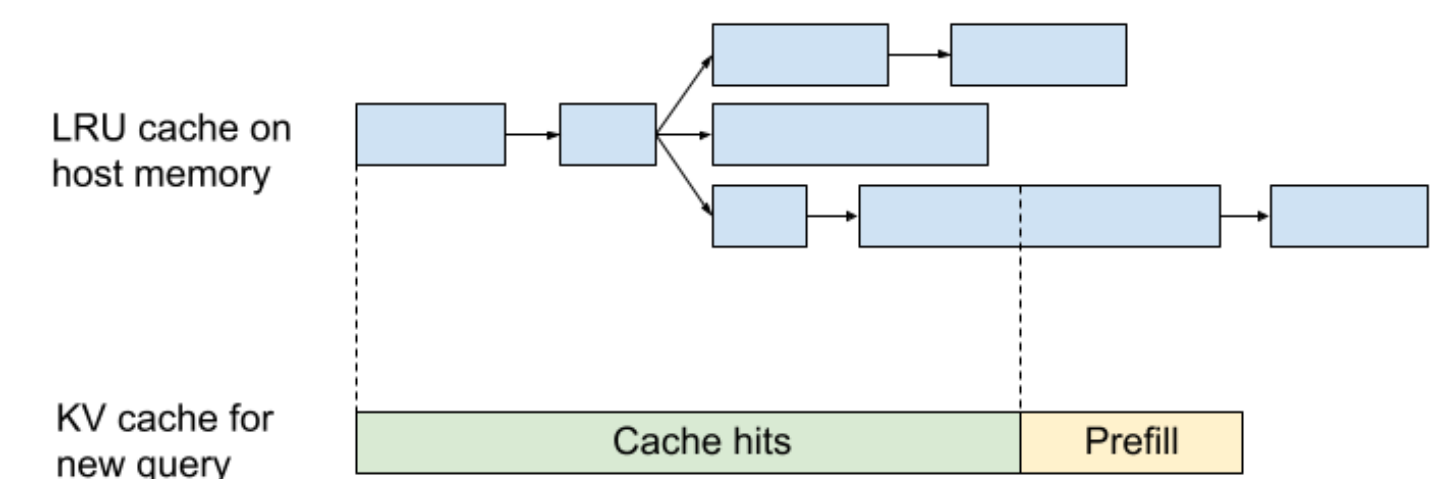


Figure: KV prefix cache implemented as an LRU trie. We can avoid duplicating KV memory by sharing prefixes. Source: [Character.ai blog](#).

Let’s look at an implementation: JetStream

Google has open-sourced a library that implements this logic called [JetStream](#). The server has a set of “prefill engines” and “generate engines”, usually on different TPU slices, which are orchestrated by a single controller. Prefill happens in the “[prefill thread](#)”, while generation happens in the “[generate thread](#)”. We also have a “[transfer thread](#)” that orchestrates copying the KV caches from the prefill to generate slices.

The Engine interface (implemented [here](#)) is a generic interface that any LLM must provide. The key methods are:

- **prefill**: takes a set of input tokens and generates a KV cache.
- **insert**: takes a KV cache and inserts it into the batch of KV caches that generate is generating from.
- **generate**: takes a set of batched KV caches and generates one token per batch entry, appending a single token’s KV cache to the decode state for each token.

We also have a PyTorch version of JetStream available [here](#).

Worked Problems

I’m going to invent a new model based on LLaMA-2 13B for this section. Here are the details:

hyperparam	value
L (num_layers)	64
D (d_model)	4,096
F (ffw_dimension)	16,384
N (num_heads)	32
K (num_kv_heads)	8
H (qkv_dim)	256
V (num_embeddings)	32,128

Question 1: How many parameters does the above model have? How large are its KV caches per token in int8? *You can assume we share the input and output projection matrices.*

► [Click here for the answer.](#)

Question 2: Say we want to serve this model on a TPUv5e 4x4 slice and can fully shard our KV cache over this topology. What’s the largest batch size we can fit, assuming we use int8 for everything and want to support 128k sequences? What if we dropped the number of KV heads to 1?

► [Click here for the answer.](#)

Question 3: How long does it take to load all the parameters into the MXU from HBM assuming they're fully sharded on a TPU v5e 4x4 slice? Assume int8 parameters. *This is a good lower bound on the per-step latency.*

► [Click here for the answer.](#)

Question 4: Let's say we want to serve this model on a TPUV5e 4x4 slice using int8 FLOPs and parameters/activations. How would we shard it for both prefill and decode? *Hint: maybe answer these questions first:*

1. What does ICI look like on a 4x4?
2. What's the roofline bound on tensor parallelism?
3. How can we shard the KV caches?

For this sharding, what is the rough per-step latency for generation?

Question 5: Let's pretend the above model is actually an MoE. An MoE model is effectively a dense model with E copies of the FFW block. Each token passes through k of the FFW blocks and these k are averaged to produce the output. Let's use E=16 and k=2 with the above settings.

1. How many total and activated parameters does it have? *Activated means used by any given token.*
2. What batch size is needed to become FLOPs bound on TPU v5e?
3. How large are its KV caches per token?
4. How many FLOPs are involved in a forward pass with T tokens?

► [Click here for the answer.](#)

Question 6: With MoEs, we can do "expert sharding", where we split our experts across one axis of our mesh. In our standard notation, our first FFW weight has shape [E, D, F] and we shard it as [E_Z, D_X, F_Y] where X is only used during training as our FSDP dimension. Let's say we want to do inference on a TPU v5e:

1. What's the HBM weight loading time for the above model on a TPU v5e 8x16 slice with Y=8, Z=16? How much free HBM is available per TPU?
2. What is the smallest slice we could fit our model on?

Question 7 [2D model sharding]: Here we'll work through the math of what the [ESTL paper](#) calls 2D weight-stationary sharding. We describe this briefly in Appendix B, but try doing this problem first to see if you can work out the math. The basic idea of 2D weight stationary sharding is to shard our weights along both the **D** and **F** axes so that each chunk is roughly square. This reduces the comms load and allows us to scale slightly farther.

Here's the algorithm for 2D weight stationary:

1. $\text{In}[B, D_X] = \text{AllGather}_{YZ}(\text{In}[B, D_{XYZ}])$
2. $\text{Tmp}[B, F_{YZ}] \{U_X\} = \text{In}[B, D_X] *_{\text{D}} W_{\text{in}}[D_X, F_{YZ}]$
3. $\text{Tmp}[B, F_{YZ}] = \text{AllReduce}_X(\text{Tmp}[B, F_{YZ}] \{U_X\})$
4. $\text{Out}[B, D_X] \{U_{YZ}\} = \text{Tmp}[B, F_{YZ}] *_{\text{F}} W_{\text{out}}[F_{YZ}, D_X]$
5. $\text{Out}[B, D_{XYZ}] = \text{ReduceScatter}_{YZ}(\text{Out}[B, D_X] \{U_{YZ}\})$

Your goal is to work out T_{math} and T_{comms} for this algorithm and find when it will outperform traditional 3D model sharding?

► [Click here for the answer!](#)

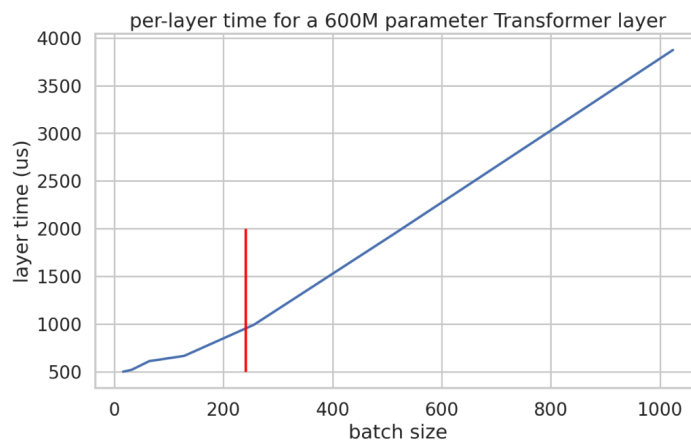
That's all for Part 7! For Part 8, with a look at how we might serve LLaMA 3 on TPUs, click [here](#).

Appendix

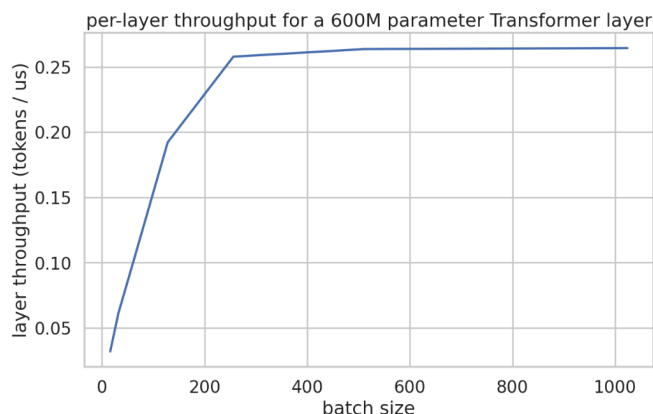
Appendix A: How real is the batch size > 240 rule?

The simple rule we provided above, that our batch size must be greater than 240 tokens to be compute-bound, is roughly true but ignores some ability of the TPU to prefetch the weights while other operations are not using all available HBM, like when doing inter-device communication.

Here's an empirical plot of layer time (in microseconds) for a small Transformer with $d_{\text{model}}=8192$, $d_{\text{ff}}=32768$, and only 2 matmuls per layer. This comes from [this Colab notebook](#). You'll see that step time increases very slowly up until around batch 240, and then increases linearly.



Here's the actual throughput in tokens / us. This makes the argument fairly clearly. Since our layer is about 600M parameters sharded 4 ways here, we'd expect a latency of roughly 365us at minimum.

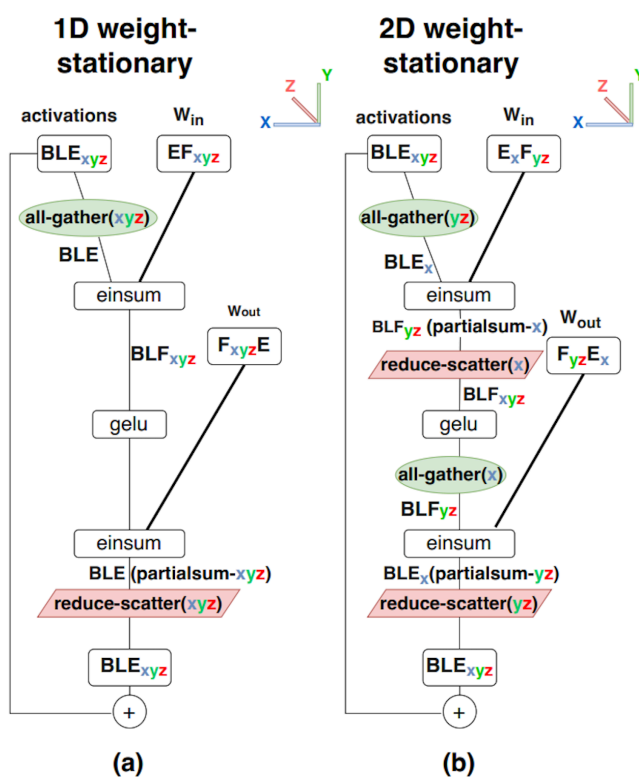


So at least in this model, we do in fact see throughput increase until about BS240 per data parallel shard.

Appendix B: 2D Weight Stationary sharding

As the topology grows, if we have access to higher dimensional meshes (like that of TPUs) it is possible to refine this further with “**2D Weight Sharding**” by introducing a second sharding axis. We call this “**2D Weight Stationary**”, and was described in more detail in the [Efficiently Scaling Transformer Inference paper](#).

Because we're only sharding the hidden F dimension in Megatron, it can become significantly smaller than E (the d_{model} dimension) once the number of chips grows large with 1D sharding. This means at larger batch sizes, it can be more economical to perform a portion of the collectives over the hidden dimension after the first layer of the MLP is applied.



This figure shows:

1. 1D weight-stationary sharding, a.k.a. Pure Megatron sharding, where activations are fully replicated after AllGather, and weights are fully sharded over the hidden F dimension.
2. 2D weight stationary sharding, where weights are sharded over both the hidden F and reduction E dimension, and activations are sharded over the E dimension. We perform an AllGather on the (yz) axis before the first layer, then ReduceScatter on the (x) axis.

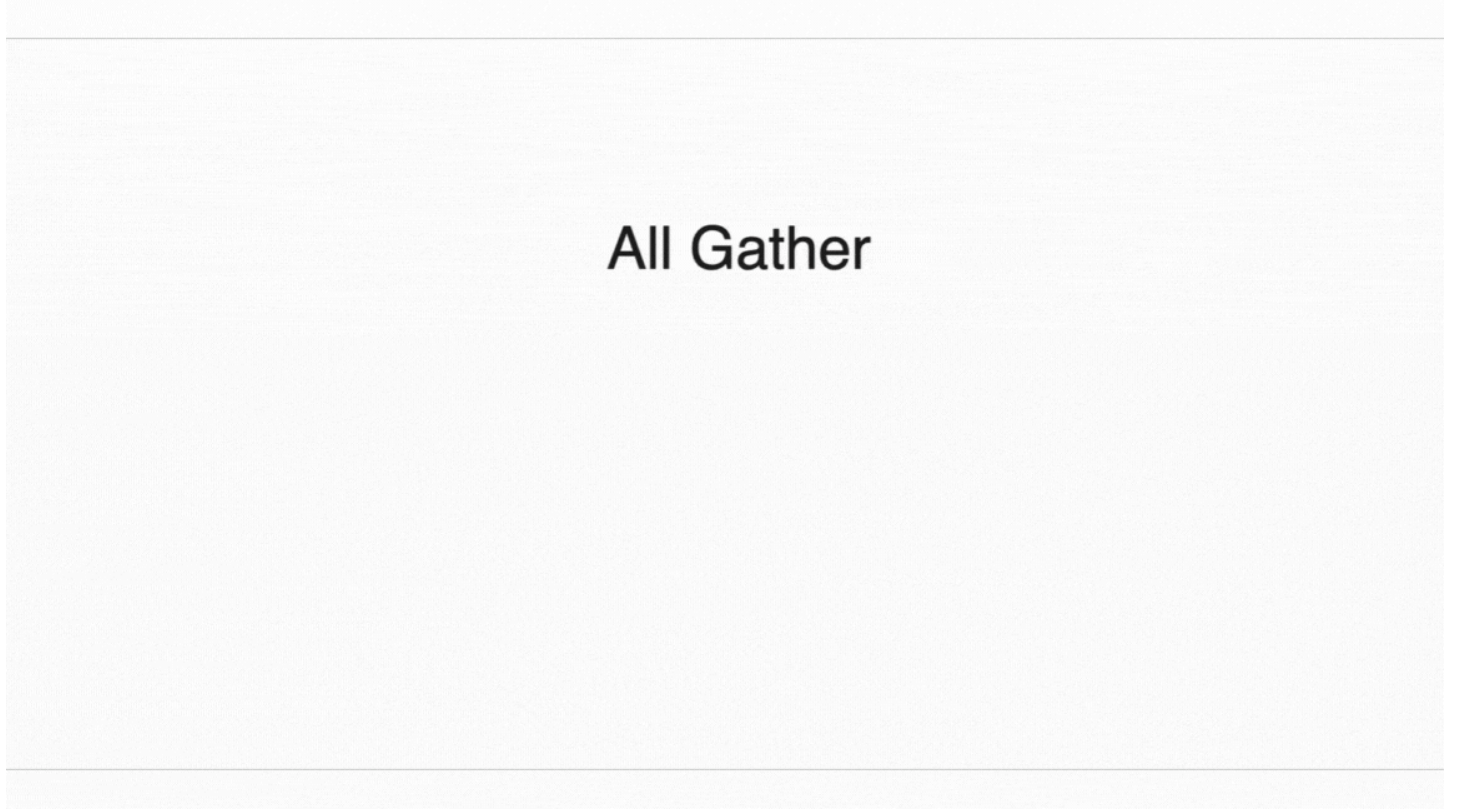
For the attention layer, Megatron style sharding is also relatively simple for smaller numbers of chips. However, Megatron happens over the n_{heads} dimension, which puts a limit on the amount of sharding that is possible. Modifying the 2D sharding for attention (instead of sharding the hidden dimension, we shard the n_{heads} dimension), we gain the ability to scale further.

Appendix C: Latency bound communications

As a recap, in [Section 3](#) we derived the amount of time it takes to perform an AllGather into a tensor of size B on each TPU, over X chips on a 1D ring with links of full-duplex bandwidth of W_{ICI} and latency T_{min} .

$$T_{total} = \max\left(\frac{T_{min} \cdot |X|}{2}, \frac{B}{W_{ICI}}\right)$$

For large B, the wall clock stays relatively constant because as you add more chips to the system, you simultaneously scale the amount of data movement necessary to perform the operation and the total bandwidth available.



All Gather

Because of the relatively low amounts of data being moved during latency optimized inference, collectives on activations are often bound by the latency term (especially for small batch sizes). One can visualise the latency quite easily, by counting the number of hops we need to complete before it is completed.

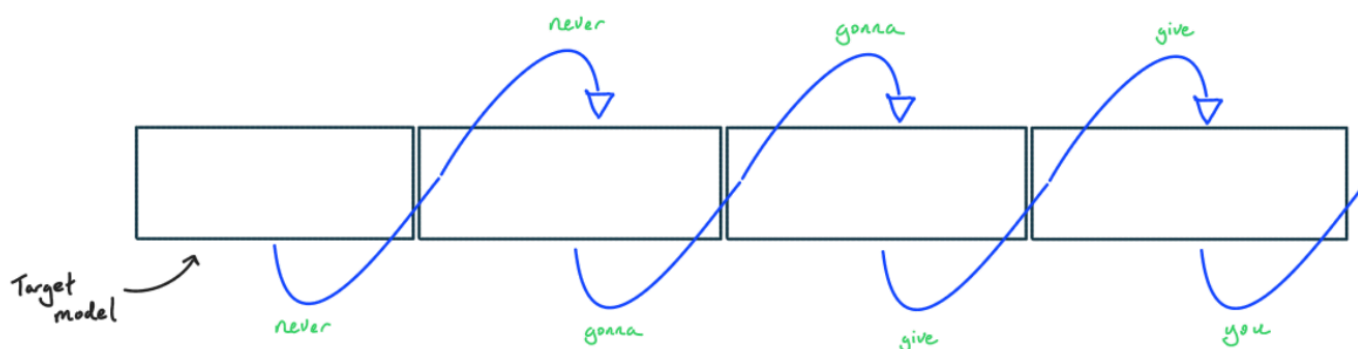
On TPUs, if the tensor size-dependent part of communication is less than 1 microsecond per hop (a hop is communication between two adjacent devices) we can be bottlenecked by the fixed overhead of actually dispatching the collective. With $4.5e10$ unidirectional ICI bandwidth, ICI communication becomes latency bound when: $(\text{bytes}/n_{\text{shards}})/4.5e10 < 1e-6$. For 8-way Megatron sharding, this is when $\text{buffer_size} < 360\text{kB}$. **This actually is not that small during inference:** with $BS=16$ and $D=8192$ in int8, our activations will use $16*8192=131\text{kB}$, so we're already latency bound.

Takeaway: our comms become latency bound when $\text{total bytes} < W_{ICI} \times 1e-6$. For instance, with model parallelism over Y , we become bound in int8 when $Y > BD/45,000$.

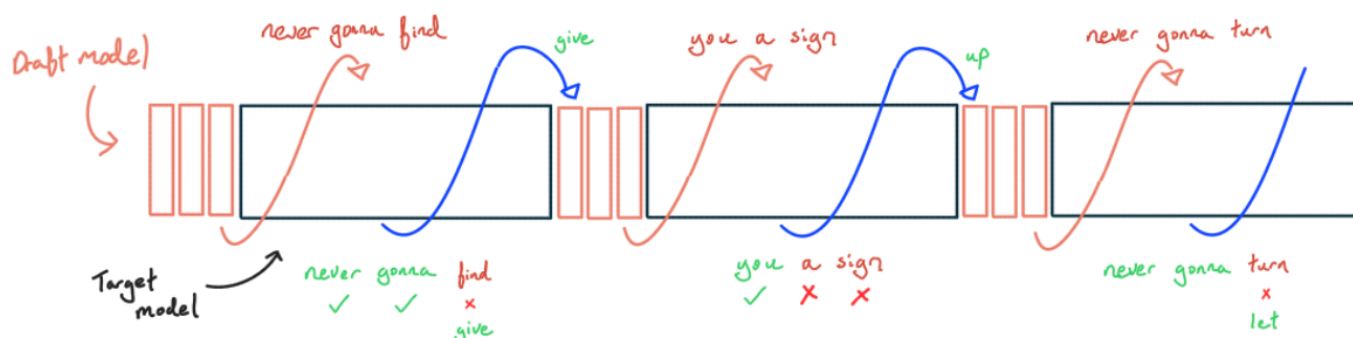
There's a parallel to be drawn here with the compute roofline — we are incurring the fixed cost of some small operations (latency for comms, memory bandwidth for matmuls).

Appendix D: Speculative Sampling

When we *really* care about end to end latency, there is one extra trick we can employ called speculative sampling [\[3\]](#) [\[4\]](#). As a recap, we usually generate tokens from a large Transformer one by one:



With speculative sampling, we use a smaller, cheaper model to generate tokens and then check the result with the big model. This is easiest to understand with *greedy decoding*.



1. We sample greedily from some smaller, cheaper model. Ideally we use a model trained to match the larger model, e.g. by distillation, but it could be as simple as simply using n-grams or token matching a small corpus of text.
2. After we've generated K tokens, we use the big model to compute the next-token logits for all the tokens we've generated so far.
3. Since we're decoding greedily, we can just check if the token generated by the smaller model has the highest probability of all possible tokens. If one of the tokens is wrong, we take the longest correct prefix and replace the first wrong token with the correct token, then go back to (1). If all the tokens are correct, we can use the last correct logit to sample an extra token before going back to (1).

Why is this a latency win? This scheme still requires us to do the FLOPs-equivalent of one forward pass through the big model for every token, but because we can batch a bunch of tokens together, we can do all these FLOPs in one forward pass and take advantage of the fact that we're *not compute-bound* to score more tokens for free.

Every accepted token becomes more expensive in terms of FLOPs on average (since some will be rejected, and we have to call a draft model), but we wring more FLOPs out of the hardware, and the small model is cheap, so we win overall. We also share KV cache loads across multiple steps, so **speculative decoding can also be a throughput win for long context**. Since everything has been checked by the big model, we don't change the sampling distribution at all (though the exact trajectory will differ for non-greedy).

Traditionally, speculative decoding relies on the existence of a smaller model with a similar sampling distribution to the target model, e.g. LLaMA-2 2B for LLaMA-2 70B, which often doesn't exist. Even when this is available, the smaller drafter can still be too expensive if the acceptance rate is low. Instead, it can be helpful to embed a drafter within the main model, for instance by adding a dedicated drafter head to one of the later layers of the base model [5] [6] [7]. Because this head shares most of its parameters with the main model, it's faster to run and matches the sampling distribution more closely.

For normal autoregressive sampling the token/s is the same as the step time. We are still beholden to the theoretical minimum step time according to the Arithmetic Intensity section here (in fact, Speculative Sampling step times are usually quite a bit slower than normal autoregressive sampling, but because we get more than 1 token out per step on average we can get much better tokens/s).

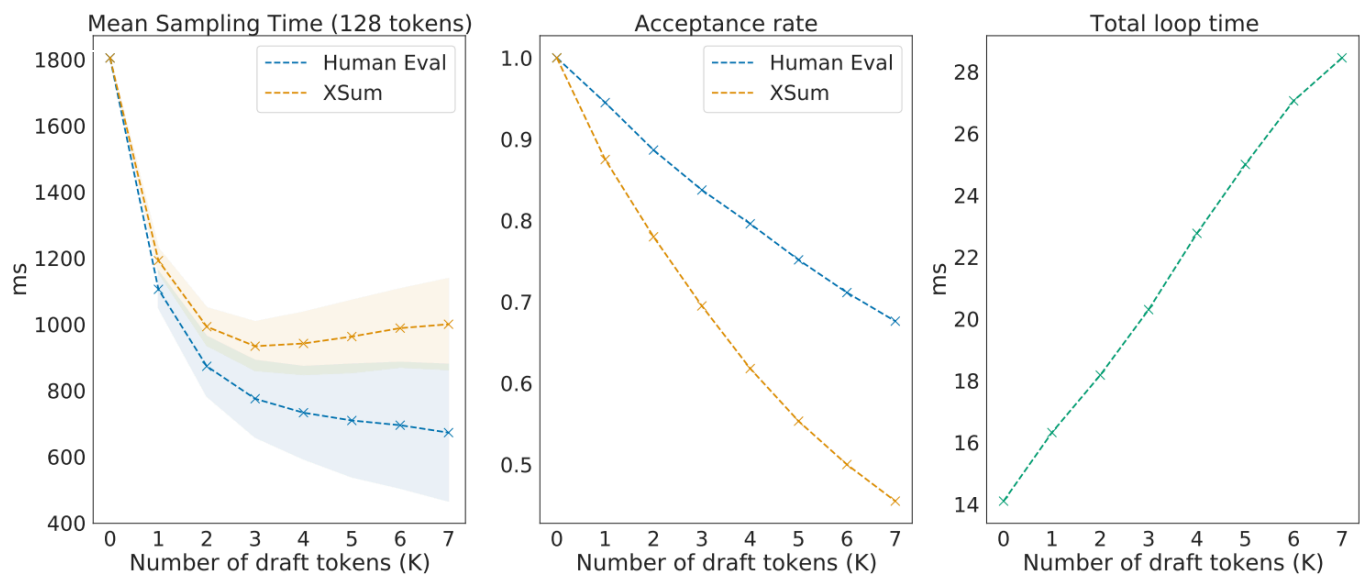


Figure 1 | Left: The average time to generate 128 tokens, with standard deviation. Note that as K increases, the overall speedup plateaus or even regresses, with XSum being optimal at $K = 3$. The variance consistently increases with K . **Middle:** The average number of tokens accepted divided by $K + 1$ – this serves as a measure of the overall efficiency of the modified rejection scheme, which decreases with the lookahead. **Right:** Average time per loop increases approximately linearly with K due to the increased number of model calls. Note that the gradient is slightly higher than the sampling speed of the draft model, due to additional overheads in nucleus decoding.

Figure: this figure shows the per-step latency and speculation success rate for Chinchilla (a 70B model from DeepMind) with a 4B parameter drafter (small model). For XSum (a natural language dataset), the ideal amount of speculation is about 3-4 tokens ahead, while HumanEval (a coding dataset) is more predictable and sees wins from more aggressive speculation.

How does this work for non-greedy decoding? This is a bit more complicated, but essentially boils down to a Metropolis-Hastings inspired algorithm where we have $P_{\text{draft model}}(\text{chosen token})$ and $P_{\text{target model}}(\text{chosen token})$ derived from the logits, and reject the chosen token probabilistically if the ratio of these probabilities is smaller than some threshold.

These [two papers](#) derived this concurrently and have good examples of how this works in practice.

Takeaway: Speculative sampling is yet another powerful lever for trading throughput for better per token latency. However, in the scenario where batch size is limited (e.g. small hardware footprint or large KV caches), it becomes a win-win.

Footnotes

1. Исторически сложилось так, что можно провести огромное количество исследований в области трансформеров, даже не затрагивая логический вывод: тесты с множественным выбором на основе скоринга можно эффективно проводить без надлежащего KV-кэша или реализации цикла генерации. Это означает, что в исследовательских кодовых базах в коде логического вывода часто можно найти много простых решений. [↗]
2. В этом разделе вы заметите, что логический вывод гораздо менее терпим к ошибкам, чем обучение. Как правило, у нас гораздо меньше операций с плавающей запятой, меньше возможностей для пакетной обработки и гораздо выше чувствительность к задержкам. Кроме того, кэши KV значительно усложняют логический вывод. [↗]
3. Здесь мы немного упрощаем, не учитывая операции с плавающей запятой, не связанные с матричным умножением, при применении функции softmax, масок и т. д. Они должны выполняться параллельно с вычислениями или чтением из НВМ, но на некоторых поколениях TPU это может быть нетривиальной задачей. Хотя эти детали не меняют главного вывода о том, что кэши KV обычно ограничены памятью, на них стоит обратить внимание. [↗]
4. В частности, благодаря функции «Мгновенное внимание», которая позволяет избежать материализации матрицы внимания [↗]
5. Случайно оставить его включенным после тренировки — простой и распространенный способ получить регрессию порядка величины [↗]
6. Под этим мы подразумеваем запуск нескольких серверов с копиями модели при меньшем размере пакета. Параллелизм данных на уровне модели — заведомо худший вариант. [↗]
7. В том смысле, что время выполнения операций с плавающей запятой не является узким местом, нам нужно беспокоиться о том, чтобы время выполнения IC1 не превышало время загрузки параметров. [↗]

References

1. **Efficiently scaling Transformer inference**
Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Heek, J., Xiao, K., Agrawal, S. and Dean, J., 2022. arXiv [cs.LG].

2. **Efficient memory management for large language model serving with PagedAttention**
Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H. and Stoica, I., 2023. arXiv [cs.LG].

3. **Fast inference from Transformers via speculative decoding**
Leviathan, Y., Kalman, M. and Matias, Y., 2022. arXiv [cs.LG].

4. **Accelerating large language model decoding with speculative sampling**
Chen, C., Borgeaud, S., Irving, G., Lespiau, J., Sifre, L. and Jumper, J., 2023. arXiv [cs.CL].

5. **EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty**
Li, Y., Wei, F., Zhang, C. and Zhang, H., 2024. arXiv [cs.LG].

6. **Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads**
Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J., Chen, D. and Dao, T., 2024. arXiv [cs.LG].

7. **DeepSeek-V3 Technical Report**
{DeepSeek-AI}, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J.L., Liang, J., Guo, J., Ni, J., Li, J., Wang, J., Chen, J., Chen, J., Yuan, J., Qiu, J., Li, J., Song, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Wang, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Wang, Q., Zhu, Q., Chen, Q., Du, Q., Chen, R.J., Jin, R.L., Ge, R., Zhang, R., Pan, R., Wang, R., Xu, R., Zhang, R., Chen, R., Li, S.S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Pan, S., Wang, T., Yun, T., Pei, T., Sun, T., Xiao, W.L., Zeng, W., Zhao, W., An, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Li, X.Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Zhang, X., Chen, X., Nie, X., Sun, X., Wang, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Song, X., Shan, X., Zhou, X., Yang, X., Li, X., Su, X., Lin, X., Li, Y.K., Wang, Y.Q., Wei, Y.X., Zhu, Y.X., Zhang, Y., Xu, Y., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Yu, Y., Zheng, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Tang, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Wu, Y., Ou, Y., Zhu, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Zha, Y., Xiong, Y., Ma, Y., Yan, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Wu, Z.F., Ren, Z.Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Huang, Z., Zhang, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Xu, Z., Wu, Z., Zhang, Z., Li, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Gao, Z. and Pan, Z., 2024. arXiv [cs.CL].

Miscellaneous

*Work done at Google DeepMind, now at MatX.

Citation

For attribution in academic contexts, please cite this work as:

Austin et al., "How to Scale Your Model", Google DeepMind, online, 2025.

or as a BibTeX entry:

```
@article{scaling-book,
  title = {How to Scale Your Model},
  author = {Austin, Jacob and Douglas, Sholto and Frostig, Roy and Levsкая, Anselm and Chen, Charlie and Vikram, Sharad
and Lebron, Federico and Choy, Peter and Ramasesh, Vinay and Webson, Albert and Pope, Reiner},
  publisher = {Google DeepMind},
  howpublished = {Online},
  note = {Retrieved from https://jax-ml.github.io/scaling-book/},
  year = {2025}
}
```